

AURIX Developer Role Upgrades: AI Feature Integrations

Team, to ensure AURIX stands out as a cutting-edge platform and to give you all strong, hands-on "AI Engineering" experience for your resumes, we are expanding your roles.

Your original role documents still apply, but we are upgrading specific features in your domains from traditional software engineering approaches to **AI-driven approaches**.

Below are the new AI components assigned to each of you. Please review your section carefully.

1. Divyanshi (IDE & Integration Engineer)



New Upgrade: AI-Powered Secret Guard & Inline Streaming

The Context: Your original instructions asked you to use "Regex" (Regular Expressions) to sweep the local code for hardcoded secrets before zipping it. Regex is prone to false positives. We are replacing this with AI.

What You Will Build:

- **Intelligent Pre-Flight Secret Guard:** You will integrate an LLM API (or a local Small Language Model) into the VS Code Extension. Before uploading the workspace, your extension will pass potential secrets to the AI to intelligently determine context (e.g., distinguishing between a real AWS key and a fake testing string `test_key_123`).
- **"Ghost-Text" Inline Patching:** Instead of just pasting the AI's suggested code patch blindly into the file, you will implement VS Code's **Inline Completion API**. This will stream the AI's suggested security fix directly into the developer's editor as "ghost text," allowing them to interactively review and accept the fix with the `Tab` key (similar to GitHub Copilot).

Why this helps your resume: You are building AI-powered developer tools natively within the IDE. Experience with SLMs and inline editor streaming (Copilot-style interactions) is incredibly rare and valuable in today's market.

2. Bhavya (Backend & Infrastructure Engineer)

New Upgrade: RAG Vector Infrastructure Node

The Context: Divyansh is building the core AI engine, which needs to perform RAG (Retrieval-Augmented Generation) to ground its logic in OWASP guidelines and past CVEs. Instead of Divyansh handling the database for this, you will own the RAG infrastructure.

What You Will Build:

- **Vector Database Setup:** You will upgrade your Supabase PostgreSQL database by enabling the `pgvector` extension. You will design and create the tables required to store high-dimensional vector embeddings.
- **Data Seeding (`seed_rag.py`):** You will take ownership of the scripts that convert raw security documentation (like OWASP guidelines) into vector embeddings (using models like HuggingFace `all-MiniLM-L6-v2`) and push them into your Supabase database.
- **Similarity Search API:** You will write the backend logic (in FastAPI/Supabase RPC) that takes a query, converts it to an embedding, and performs a cosine similarity search against the vector database to return the most relevant threat intelligence.

Why this helps your resume: You are no longer just building standard CRUD APIs. You are architecting the foundational database infrastructure required for modern AI RAG pipelines.

3. Bhumika (Web Frontend Developer)

New Upgrade: AI Executive Summary Generation & Client-Side LLM Orchestration

The Context: Originally, your role was to fetch the verified vulnerabilities from Bhavya's API and display them in a Kanban board. We are upgrading your dashboard to be an active AI participant.

What You Will Build:

- **AI Executive Summary Engine:** When a user opens their scan results, you will not just show the raw data. You will take the JSON payload, feed it into a prompt, and use an AI framework (like `LangChain.js` or `Vercel AI SDK`) to dynamically generate and stream a conversational "Threat Landscape Report" directly in the UI.
- **Streaming & State Management:** You will handle the complexities of streaming AI text (the typewriter effect) and managing conversational memory for the Contextual AI Chat Assistant, allowing the user to have a continuous dialogue about their vulnerabilities.

- **Prompt Engineering:** You will write the client-side system prompts that instruct the LLM on how to format and present the security data to the user.

Why this helps your resume: You are transitioning from a standard React developer to an AI Application Developer, gaining highly sought-after skills in client-side AI orchestration, streaming, and prompt engineering.

Comprehensive Updated Component Checklist

For clarity, here is the complete, updated list of components each of you will now build (combining your original tasks with your new AI upgrades).

Bhavya (Backend & Infrastructure Engineer)

- **API Gateway:** REST APIs for routing web and VS Code client requests.
- **The Auth Vault:** Supabase Auth integration and secure JWT session management.
- **PostgreSQL Database:** Core tables for users, projects, and scan history.
- **Redis Queue:** Shock-absorber queue to manage asynchronous AI execution.
- **Automated Data Sanitizer:** Storage cleanup via TTL/Cron jobs.
- **Rate Limiter:** API quota enforcement to protect compute resources.
-  **RAG Vector Database:** Supabase **pgvector** tables and embeddings storage.
-  **Threat Intel API:** Backend logic for similarity search and data seeding.

Bhumika (Web Frontend Developer)

- **Authentication Screens:** Login and signup UI ("The Door").
- **Dual-Mode Ingestion Form:** UI for GitHub URL pasting and OAuth syncing.
- **Real-Time Status UI:** Polling mechanism and progress bars.
- **Triage Kanban Board:** Data-grid to filter and manage active threats.
- **Vulnerability Detail View:** Syntax highlighting for PoCs and patch code.
- **Automated Remediation Module:** One-Click PR integration via GitHub API.
-  **Contextual AI Chatbot:** Interactive security tutor using Vercel AI SDK/LangChain.js.
-  **Executive Summary Engine:** Client-side LLM streaming for conversational threat landscape reports.

Divyanshi (IDE & Integration Engineer - VS Code)

- **VS Code Auth Module:** Native login prompt and SecretStorage API token management.
- **Workspace Packager:** "Zip & Ship" local file system logic (respecting **.gitignore**).
- **API Connector:** Multipart/form-data upload and polling status logic.
- **IDE Diagnostics UI:** Editor highlights (red squiggly lines) and tooltips for vulnerabilities.
- **Attack Path Webview:** Rich HTML panel for full exploit chain reports.

- 🚀 **AI-Powered Secret Guard:** SLM/LLM-based local scan to intelligently prevent sensitive data leaks.
- 🚀 **"Ghost-Text" Auto-Patcher:** Inline code completion API to stream AI fixes directly into the editor.

Divyansh (Core Engine Architect - AI & Sandbox)

- **Triage Node & Semantic Slicer:** AST context extraction.
- **AurixState & LangGraph Orchestrator:** The core AI state machine and execution loop.
- **Consensus Gate & Agents:** Prompt engineering for Logic, Red, and Blue Agents.
- **Reflexion Engine:** Self-healing retry loop for broken exploit scripts.
- **Ephemeral Docker Runner:** Network-isolated sandbox for safely executing AI exploits.